# ARL Spec: On-demand Water Treatment

Brendan Griffiths

## Contents

1 Service overview . . . . . 2
1.1 General state and routines . . . . . 2
1.2 Message conventions . . . . . 2
2 Transmitted data . . . . . 3
2.1 Telemetry (`arl/owt/<entity>/telemetry/water`, every 1,800 s) . . . . . 3
2.2 State (retained) . . . . . 4
2.3 Events . . . . . 4
3 Controls . . . . . 5
4 Alerts . . . . . 5
5 Configuration . . . . . 6
6 Scenario events . . . . . 7

# 1 Service overview

The On-demand Water Treatment (OWT) service recycles the laboratory's water. Used (grey) water collects in the intake tank, the treatment plant cleans it, and the clean water returns to storage for reuse. Topic prefix: `arl/owt`.

| Entity | Kind | Plant |
|---|---|---|
| `global` | `service` | The plant as a whole: water consumed, power used, recycling efficiency. |
| `intake` | `tank` | Greywater collection tank, 20,000 L. |
| `treatment` | `plant` | Filtration, UV and chemical dosing, 0 to 600 L/h. |
| `storage` | `tank` | Clean water storage, 50,000 L. |

## 1.1 General state and routines

- **Reporting**: the plant is simulated every minute but reports every 1,800 s (30 min) on each entity's `water` stream. This is the slowest service at the ARL.
- **Water cycle**: about 60 residents use clean water from storage, with peaks in the morning and the evening. Most of it returns to the intake as greywater; the rest is lost. The treatment plant pumps greywater from the intake at its cleaning rate (default set point 350 L/h), returns most of it to storage as clean water and rejects the rest as brine.
- **Diverting the intake**: when the intake is diverted, greywater goes to the evaporation pond instead of the intake. This protects the plant from contaminated water, but the water is lost.
- **Storage reserve**: the plant keeps a minimum clean volume in storage. When clean water runs out, the borehole supplies the residents instead; that water is consumed but not recycled.
- **Filters**: filter health falls as water is treated. Dirty filters reduce the cleaning rate and the water quality. A backwash (30 min, treatment paused, uses 500 L of clean water) restores much of the filter health.
- **Water quality**: treated water should be clear (low turbidity) and near neutral pH. Running the plant hard (above about 450 L/h) or with dirty filters makes the water cloudier.
- **Recycling efficiency**: clean water returned to storage over water consumed, over the trailing 24 h.
- **Load**: 4 messages every 30 minutes, plus state changes, backwash events and alerts.

## 1.2 Message conventions

Every service follows the same MQTT layout. `<entity>` is an entity id (or `global` for the whole service).

| Topic | Retained | Content |
|---|---|---|
| `arl/owt/$topology` | yes | Machine-readable list of the entities, streams (with field names, types, units and periods), state items, events and controls of the service. Re-published when it changes. |
| `arl/owt/<entity>/telemetry/<stream>` | no | Periodic measurements. |
| `arl/owt/<entity>/state/<item>` | yes | The current value of a state item, re-published when it changes. |

| Topic | Retained | Content |
|---|---|---|
| `arl/owt/events/<kind>` | no | One-off events, including `alert`. |
| `arl/owt/cmd/<entity>/<control>` | no | Commands sent by the platform. |
| `arl/owt/ack/<command_id>` | no | Acknowledgements of a command. |

Telemetry and event payloads carry an envelope: `v` (schema version, integer, currently 1), `ts` (simulated time, Unix milliseconds), `seq` (per-topic sequence number, restarts with the service; gaps reveal lost messages) and `entity` (entity id). The measurement fields are added next to the envelope. A measurement that cannot be taken is `null`. State payloads are `{"v", "ts", "entity", "item", "value"}`.

**Commands** are JSON `{"command_id": <string>, "researcher": <researcher code>, "value": <value>}`. The researcher must be verified by the ARL API for the service. Acknowledgements carry `command_id`, `target`, `control`, `researcher`, `state` (`accepted`, `rejected`, `in_progress`, `completed`, `failed`), `progress` (0 to 1 for long-running controls), `reason` and `detail`. Rejection and failure reasons: `controls_disabled`, `malformed`, `unknown_control`, `unknown_target`, `unknown_researcher`, `researcher_defunct`, `researcher_not_permitted`, `api_unavailable`, `invalid_value`, `precondition_failed`, `busy` (the target is already running a long command), `fault`, `timeout`, `interrupted`.

**Alerts** are events on `arl/owt/events/alert` with `alert_id`, `type` (the alert key below), `severity` (`info`, `warning`, `critical`), `state` (`raised` or `cleared`), `value`, `limit`, `unit`, `message` and `metadata`. An alert that escalates is raised again with the higher severity under the same `alert_id`. Limits come from the ARL API (`/v1/alert-limits/` + `owt`).

Every service also publishes these:

| Measurement | Key | Level | Type / unit | Description |
|---|---|---|---|---|
| Online | `online` | Global | boolean | Retained state. `true` while the service runs, `false` when it stops or loses its broker connection (last will). |
| Active alerts | `active_alerts` | Global | array of alert ids | Retained state. The ids of the alerts currently raised, in the order they were raised. |

# 2 Transmitted data

## 2.1 Telemetry (`arl/owt/<entity>/telemetry/water`, every 1,800 s)

| Measurement | Key | Level | Type / unit | Description |
|---|---|---|---|---|
| Unclean volume | `unclean_volume_l` | Entity `intake` | integer, litres | Greywater waiting in the intake tank. |
| Cleaning rate | `cleaning_rate_lph` | Entity `treatment` | integer, L/h | Rate at which the plant is currently treating water. 0 when disabled, backwashing, faulted or the intake is empty. |
| Turbidity | `turbidity_ntu` | Entity `treatment` | float, NTU | Cloudiness of the treated water (lower is cleaner). |

| Measurement | Key | Level | Type / unit | Description |
|---|---|---|---|---|
| pH | `ph` | Entity `treatment` | float, pH | Acidity of the treated water (7 is neutral). |
| Pump status | `pump_status` | Entity `treatment` | enum | `off`, `running` or `fault`. |
| Filter health | `filter_health_pct` | Entity `treatment` | float, % | Condition of the filters (100% is clean). |
| Clean volume | `clean_volume_l` | Entity `storage` | integer, litres | Clean water in storage. |
| Water consumed | `consumed_l` | Global | integer, litres | Water used by the residents since the previous report. |
| Power | `power_w` | Global | integer, watts | Electrical power used by the plant (UV and pumps). |
| Recycling efficiency | `recycling_efficiency_pct` | Global | float, % | Clean water returned to storage over water consumed, over the last 24 h. |
| Heartbeat | `ts` (envelope) | Each entity | timestamp, ms | Time of the entity's latest report. |

## 2.2 State (retained)

| Measurement | Key | Level | Type / unit | Description |
|---|---|---|---|---|
| Treatment enabled | `enabled` | Entity `treatment` | boolean | Whether the treatment plant is enabled. |
| Cleaning rate set point | `rate_setpoint_lph` | Entity `treatment` | integer, L/h | The cleaning rate the plant aims for. |
| Intake diverted | `diverted` | Entity `intake` | boolean | Whether greywater is being diverted to the evaporation pond. |
| Storage reserve | `reserve_l` | Entity `storage` | integer, litres | Minimum clean volume kept in storage. |

## 2.3 Events

| Measurement | Key | Level | Type / unit | Description |
|---|---|---|---|---|
| Backwash | `arl/owt/events/backwash`: `event` | Entity `treatment` | enum | A filter backwash `started` or `completed`. |
| Alert | `arl/owt/events/alert` | Global / entity | alert object | Alert raised or cleared (see *Alerts*). |

# 3 Controls

| Control | Key | Level | Value (type / unit) | Description |
|---|---|---|---|---|
| Enable Treatment | `enable_treatment` | Entity `treatment` | boolean | Turns the treatment plant on or off (pumps `off` when disabled). |
| Set Cleaning Rate | `cleaning_rate` | Entity `treatment` | integer, L/h, 0 to 600 | Sets the cleaning rate set point. High rates lower the water quality and wear the filters faster. |
| Filter Backwash | `filter_backwash` | Entity `treatment` | boolean, must be `true` | Cleans the filters: 30 min `in_progress` (treatment paused), then `completed` with detail `filters_restored`. Uses 500 L of clean water, so it is rejected when storage holds less (`low_storage`). |
| Divert Intake | `divert_intake` | Entity `intake` | boolean | Diverts greywater to the evaporation pond (lost) instead of the intake. |
| Set Storage Reserve | `storage_reserve` | Entity `storage` | integer, litres, 0 to 40,000 | Minimum clean volume to keep in storage. Below it the borehole supplies the residents. |

# 4 Alerts

| Alert | Key | Level | Severity, limit key (default, unit) | Description |
|---|---|---|---|---|
| Low Clean Water | `low_clean_water` | Entity `storage` | warning; the storage reserve (`reserve_l` state, L) | Clean volume below the reserve (`metadata.reserve_l`). Clears 2,000 L above it. |
| Intake Full | `intake_full` | Entity `intake` | warning; `owt.intake.max_pct` (95 % of capacity) | Intake volume above the limit. `metadata.diverted`. Clears 10 points below the limit. |
| Water Quality | `water_quality` | Entity `treatment` | critical; `owt.quality.max_ntu` (1.0 NTU), `owt.quality.ph_min` (6.5), `owt.quality.ph_max` (8.5) | A report's turbidity above the limit or pH outside the limits (the other reading is in `metadata`). Clears after two good reports in a row. |
| Filter Clogged | `filter_clogged` | Entity `treatment` | warning; `owt.filter.min_pct` (20 %) | Filter health below the limit. Clears 10 points above it. |
| Pump Fault | `pump_fault` | Entity `treatment` | critical; no limit | The treatment pump has failed. Clears when it recovers. |
| Low Recycling Efficiency | `low_recycling_efficiency` | Global | info; `owt.recycling.min_pct` (70 %) | Recycling efficiency below the limit in two reports in a row. `metadata.diverted` and `metadata.clean_volume_l`. Clears 5 points above the limit. |

# 5 Configuration

The service reads a TOML file (`config/` + `owt.toml` in the bundle, `--config` or `ARL_CONFIG` for a binary). Environment variables override the file, and command-line options override both (see *Running the ARL*). Defaults are the values in the provided file.

### 5.0.1 Common keys

| Key | Type / unit | Default | Description |
|---|---|---|---|
| `seed` | integer | `1` | Random seed. The same seed, start time, scenario and commands give the same data. Env `ARL_SEED`. |
| `speedup_multiplier` | float, × | `1.0` | Simulation speed relative to real time, up to the service's `max_speedup`. Env `ARL_SPEEDUP`. |
| `control_enabled` | boolean | `true` | Whether the service accepts commands. When `false` every command is rejected (`controls_disabled`). Env `ARL_CONTROL_ENABLED`. |
| `start_time` | `"now"` or RFC 3339 | `"now"` | Simulated time at start-up. Env `ARL_START_TIME`. |
| `scenario` | string, path | `""` | Scenario file to load, empty for none. Env `ARL_SCENARIO` (compose file). |
| `mqtt.address` | string | `"mosquitto"` | MQTT broker host. Env `MQTT_BROKER_ADDRESS`. |
| `mqtt.port` | integer | `1883` | MQTT broker port. Env `MQTT_BROKER_PORT`. |
| `mqtt.user` | string | `""` | Broker username, empty for anonymous. Env `MQTT_BROKER_USER`. |
| `mqtt.password` | string | `""` | Broker password. Env `MQTT_BROKER_PASSWORD`. |
| `mqtt.queue_capacity` | integer, messages | `200000` | Outgoing messages the service buffers while the broker is slow or unreachable. |
| `api.address` | string | `"api"` | ARL API host. Env `ARL_API_ADDRESS`. |
| `api.port` | integer | `8080` | ARL API port. Env `ARL_API_PORT`. |
| `api.require_researcher` | boolean | `true` | Whether commands must carry a researcher code the API accepts for this service. |
| `api.researcher_cache_ttl_s` | integer, s | `60` | How long a researcher check is remembered. |
| `api.alert_limits_refresh_s` | integer, s | `300` | How often the alert limits are re-read from the API. |
| `health.port` | integer | `9100` | Port of the service's `/healthz` endpoint (used by `healthcheck`). Env `HEALTH_PORT`. |

### 5.0.2 OWT keys (`[owt]`)

| Key | Type / unit | Default | Description |
|---|---|---|---|
| `frequency_hz` | float, Hz | `0.000556` | Reporting rate of the `water` streams (0.000556 Hz is every 30 minutes). |
| `residents` | float, people | `60` | Number of residents using water. |
| `litres_per_person_day` | float, L | `120` | Water use per resident per day. |
| `intake_capacity_l` | float, L | `20000` | Size of the intake tank. |
| `storage_capacity_l` | float, L | `50000` | Size of the clean storage tank. |

| Key | Type / unit | Default | Description |
|---|---|---|---|
| `initial_intake_l` | float, L | `10000` | Greywater in the intake at start-up. |
| `initial_storage_l` | float, L | `30000` | Clean water in storage at start-up. |
| `initial_filter_pct` | float, % | `70` | Filter health at start-up. |
| `initial_reserve_l` | float, L | `5000` | Storage reserve at start-up. |
| `initial_rate_lph` | float, L/h | `350` | Cleaning rate set point at start-up. |
| `max_speedup` | float, × | `500` | Highest `speedup_multiplier` the service accepts. |

# 6 Scenario events

Events with `service = "owt"` (format in *Running the ARL*).

| Fault | Entity | Params (default) | Effect |
|---|---|---|---|
| `pump_fault` | `treatment` | none | The treatment pump fails and treatment stops (Pump Fault). |
| `intake_contamination` | `intake` | `ntu` (3.5) | Dirty water reaches the intake for `duration` (default 4 h): treated water turns cloudy unless the intake is diverted (Water Quality). |
| `dosing_fault` | `treatment` | `towards_ph` (9.0) | Chemical dosing fails and the pH drifts towards `towards_ph` (Water Quality). |
| `leak` | `storage` | none | Clean storage leaks water. |
| `sensor_drift` | `treatment` | none | The turbidity sensor reads low until the next backwash. |
| `filter_replacement` | `treatment` | none | New filters: filter health returns to 100%. |
| `set_state` | any | `filter_pct`, `storage_l`, `intake_l` | Sets the filter health, clean volume and/or intake volume directly. |
| `repair` | `intake`, `treatment` or `storage` | none | Ends the faults on that entity. |

---

**AI Declaration**: Claude Code (Opus 5.5) - generated from the arl-sim source, reviewed